QUESTION

Where does the operator go in **infix**, **prefix**, and **postfix**? Write `3 + 4` in all three.

QUESTION

Write `(3 + 4) * 2` in postfix and in prefix. What do you notice about parentheses?

QUESTION

State the rule for evaluating a postfix expression with one stack.

QUESTION

Evaluating postfix and converting infix to postfix each use a stack. What does each stack hold?

QUESTION

Evaluate `3 4 2 * +` with a stack. What is on the stack after each token? (Write it bottom → top, left to right.)

QUESTION

When an operator pops two values, which one is the **left** operand? Evaluate `8 2 / 3 -`.

ANSWER

Infix: between the operands: `3 + 4`.

Prefix: before them: `+ 3 4`.

Postfix: after them: `3 4 +`.

ANSWER

Postfix: `3 4 + 2 *`. Prefix: `* + 3 4 2`.

Neither needs **parentheses or precedence rules**: the order of the tokens already says what to do first.

ANSWER

Read tokens left to right.

Number: push it.

Operator: before popping, require **at least two values** (otherwise the expression is invalid). Then pop the **right** operand `b`, then the **left** operand `a`, and push `a op b`.

At the end, exactly **one** value should be left: that is the answer.

ANSWER

Evaluation stack: numbers (values), where an operator pops two and pushes one.

Conversion stack: operators (and `(`) that are waiting to be written to the output.

ANSWER

`[3]`, `[3 4]`, `[3 4 2]`; then `*` pops 2 and 4 and pushes 8: `[3 8]`; then `+` pops 8 and 3 and pushes 11: `[11]`.

The value is **11**.

ANSWER

The **first** value popped is the **right** operand; the second is the left.

`8 2 /` gives $8/2 = 4$; then `4 3 -` gives $4 - 3 = \mathbf{1}$.

Matters for `-` and `/`, not for `+` and `*`.

QUESTION

A student treats the first popped value as the **left** operand. Using the lecture's C++ integer arithmetic, what does the code give for `8 2 / 3 -`?

QUESTION

Why do we **tokenize** before parsing? What should `tokenize("12+3*(4-1)")` return?

QUESTION

`evalPostfix` uses `std::stack<int>`. What does that mean for `/`? What about dividing by zero?

QUESTION

The input assumptions say literals are nonnegative. Can a result be negative? Evaluate `0 7 - 2 /`.

QUESTION

What do the C++ examples in class assume about the input?

QUESTION

What do the **leaves** and **inner nodes** of an expression tree represent?

ANSWER

Single characters are not enough: `12` must stay **one token**.

```
{"12", "+", "3", "*", "(", "4", "-", "1", ")"}
```


ANSWER

`/` computes $2/8 = 0$ in integer arithmetic; then `-` computes $3 - 0 = \mathbf{3}$.

The correct answer is 1.

ANSWER

Yes: negative intermediate results are allowed. Unary minus in the **input** (like `-7`) is outside the lecture parser's scope.

`0 7 -` gives -7; then $-7/2 = \mathbf{-3}$ (integer division truncates toward zero). Infix: `(0 - 7) / 2`.

ANSWER

It is **integer arithmetic**. Division truncates toward zero: `7 / 2` is 3 and `-7 / 2` is -3.

Dividing by zero is **not permitted**; a complete evaluator must check for it.

ANSWER

Leaves are numbers.

Each **inner node** is an operator applied to its two subtrees.

The tree for `3 + 4 * 2` has `+` at the root, with `3` on the left and `*` (children `4`, `2`) on the right.

ANSWER

Valid expressions with **nonnegative integer literals**, binary operators `+ - * /`, and **matching parentheses**.

A complete evaluator must validate its input.

QUESTION

Which traversal of an expression tree gives **postfix**? **Prefix**? **Infix**?

QUESTION

Draw the expression trees for `3 + 4 * 2` and `(3 + 4) * 2`. Which node is the root of each? Read each in post-order.

QUESTION

Compare `3 + 4 * 2` with `3 4 2 * +`. Did the numbers change order? Which operator comes out first?

QUESTION

Where does the name **shunting-yard** come from? What are the two containers?

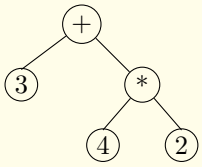
QUESTION

What does shunting-yard do with a **number** token? With the **end** of input?

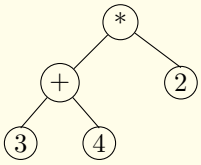
QUESTION

State the rule for an **operator** token `o`.

ANSWER



root `+`
post-order: `3 4 2 * +`



root `*`
post-order: `3 4 + 2 *`

Both expressions are infix. Parentheses change the grouping, so the tree and the postfix order change.

ANSWER

Post-order (left, right, node): postfix.

Pre-order (node, left, right): prefix.

In-order: infix; add parentheses **where needed** to preserve the tree's grouping.

ANSWER

A railway shunting yard: cars wait on a side track and are released in the order the schedule needs.

The algorithm has an **output list** and an **operator stack**.

ANSWER

The numbers keep their order, so they go straight to the output.

`+` was written first but comes out **last**; `*` comes out first. Operators need a place to **wait**: a stack, because the newest waiting operator is released first.

ANSWER

While the stack is nonempty, its top is an operator, and that operator's precedence is at least that of `o`: pop it to the output.

Then **push** `o`.

ANSWER

A number is appended to the **output** at once.

At the end, **pop the remaining operators** to the output. A `(` still on the stack means **mismatched parentheses**.

QUESTION

What do the tokens `(` and `)` do?

QUESTION

What precedence values do the operators have, and what prevents an operator from popping a `(`?

QUESTION

Why does the pop test use $\geq$ (equal precedence) and not just $>$?

QUESTION

Convert `3 + 4 * 2` token by token. Give the output and the stack (bottom $\rightarrow$ top) after each token.

QUESTION

Convert `3 + 4 * 2`. When `*` arrives, why is nothing popped?

QUESTION

Convert `(3 + 4) * 2`. What does the `)` do?

ANSWER

`*` and `/` are 2; `+` and `-` are 1.

The loop tests `isOp(ops.top())`, which is **false** for `(`, so an operator never pops it. Only the matching `)` removes it.

ANSWER

`(` : push it.

`)` : pop operators to the output until `(` is on top, then **discard** the `(`. If the stack empties first: **error**.

Neither parenthesis goes to the output. The `(` is a **wall**: only its own `)` removes it.

ANSWER

Token	Output	Stack
`3`	`3`	empty
`+`	`3`	`+`
`4`	`3 4`	`+`
`*`	`3 4`	`+ *`
`2`	`3 4 2`	`+ *`
end: pop `*`	`3 4 2 *`	`+`
end: pop `+`	`3 4 2 * +`	empty

Result: `3 4 2 * +`.

ANSWER

The operators are **left-associative**: `8 - 3 - 2` means $(8 - 3) - 2$.

With `>`, the output would be `8 3 2 - -` $= 7$. Correct: `8 3 - 2 -` $= 3$.

ANSWER

`)` pops `+` to the output and discards the `(`.

The `(` forced the `+` out **before** `*` arrived.

Result: `3 4 + 2 *`.

ANSWER

`*` has **higher** precedence than the `+` on top, so the `+` keeps waiting.

Result: `3 4 2 * +`. At the end, `*` is popped before `+`.

QUESTION

Convert `5 + (1 + 2) * 4 - 3`. What does the final `-` do?

QUESTION

Convert `6 + 2 * (5 - 1)` to postfix and evaluate it.

QUESTION

Name three mistakes in shunting-yard code.

QUESTION

What can go wrong with mismatched parentheses? Why is that dangerous in C++?

QUESTION

What are the three stages of the Project 4 evaluator?

QUESTION

What is the largest number of values on the evaluation stack while evaluating `3 4 2 * +`?

ANSWER

Postfix: `6 2 5 1 - * +`.

The `-` makes 4, the `*` makes 8, and the `+` makes $6 + 8 = \mathbf{14}$.

ANSWER

The new `-` pops `*` (higher), then `+` (equal), then stops because the stack is empty. Finally the incoming `-` is **pushed**.

Result: `5 1 2 + 4 * + 3 -` = **14**.

ANSWER

A `)` with no `(` empties the stack, or a `(` is left at the end.

Calling `ops.top()` on an **empty stack** is **undefined behavior**, so real code must detect both cases.

ANSWER

Popping only on `>`, so `8 - 3 - 2` gives 7.

Forgetting the final flush: operators left on the stack must be output.

Letting an operator pop a `(`, or tokenizing `12` as two characters.

ANSWER

3: after reading `3`, `4`, `2` the stack is `[3 4 2]`. Then `*` leaves `[3 8]` and `+` leaves `[11]`.

ANSWER

1. **Tokenize** the infix text.
2. **Convert** the tokens to postfix (shunting-yard).
3. **Evaluate** the postfix expression with a stack.